Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»

FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»

Field of Study:
«Computer Science»

Тема	**Исследование и адаптация компонентов персистентной Phantom ОС на Genode**
Topic	**Research and adaptation of components from the persistent OS Phantom for Genode**

Работу выполнил / *Prepared by*	**Воронин Михаил Сергеевич / Mikhail Voronin**	подпись / signature
Руководитель выпускной квалификационной работы / *Final Qualification Work Supervisor*	**Бурмяков Артем Сергеевич / Artem Burmyakov**	подпись / signature
Консультанты *Consultants*	**Тормасов Александр Геннадиевич / Alexander Tormasov**	подпись / signature

Иннополис, Innopolis, 2026

Contents

List of Tables

List of Figures

Abstract

For decades, the evolution of operating systems has been shaped by a proven conceptual model that emerged in the mid-20th century with the advent of UNIX. Today, however, this model is increasingly revealing its fundamental limitations. Attempts to circumvent these limitations as early as the late 20th century led, among other things, to the emergence of the concept of operating systems with persistent memory. Despite the merits of this paradigm, research was effectively halted due to hardware limitations. Since then, computer performance has increased significantly, giving impetus to new work in this direction. A striking illustration of this is the Phantom OS. However, writing and implementing a completely new operating system is a lengthy, labor-intensive process that entails a large number of errors. Mitigating these consequences is aided by the use of proven mechanisms, which is facilitated by porting to Genode. In this paper, we examine the process of adapting PhantomOS components for Genode. The issue of secure snapshot storage remains relevant. To address this, file_vault was adapted to perform block-level encryption and provide a file system session. This led to the creation of se_vault. However, the downside of this security measure, which limits its usability, is a significant performance degradation—approximately 24 times slower—due to the use of the experimental Tresor library. These results open up avenues for further research in the field of persistent operating system security and may contribute to their wider adoption.

Chapter 1

Introduction

1.1 Background

Today, a widely used concept involves dividing data into two categories: active and stored. In this model, the programmer is responsible for moving data between these states. In the context of this paper, this approach will be referred to as a "two-level store." The obvious drawbacks of this approach include:

- Data loss due to unexpected shutdowns or process termination;
- Increased development effort (and higher cost);

This approach is contrasted with the concept of orthogonal persistence, discussed by Atkinson and Morrison in 1990 [1]. It entails that data is constantly accessible and does not require explicit actions on the part of the programmer to save it to the drive. Instead, the responsibility for this falls on the environment in which the program is executed. This reduces the amount of data manipulation, which typically accounts for up to 30% of the code [1]. The number of errors when writing such programs is also significantly reduced. One of the less obvious

advantages of this approach is the highly efficient use of disk bandwidth [2]. Over time, several operating systems have emerged that implement this concept. These include KeyKOS (EROS and Coyotos), Multics (with single-level storage), and others. All of these systems are quite old, and research in this area came to a standstill after their introduction. However, significant advances in hardware capabilities have reignited interest in this field, leading to the development of the Aurora operating system (transparent persistence) [3]. The PhantomOS project is one such modern system that implements the concept of orthogonal persistence.

1.2 PhantomOS

According to the developers themselves, Phantom OS is an operating system that implements the principles of orthogonal persistence within the Phantom Virtual Machine (PVM) [4], [5]. According to the documentation, the system guarantees recovery even in the event of an unexpected reboot, provided the consistent data is not too old. This is achieved through the use of a snapshot mechanism: from time to time, the system saves the PVM's state to disk without interrupting its operation, which is accomplished using copy-on-write (CoW). At present, Phantom is described by its developers as a proof-of-concept (PoC) for the concept of orthogonal persistence. Many mechanisms and subsystems have been implemented, and their functionality has been verified to some extent, but they have not been fully tested. The current level of stability does not allow the OS to be used for industrial purposes, and work on refining it is still ongoing. One area of this work involves porting PhantomOS to the Genode framework. This should accelerate development and help avoid a large number of errors by using proven solutions, as well as positively impact system security through improved isolation, which is

enabled by the use of capability-based security.

1.3 Genode

The Genode OS Framework is a set of tools for building operating systems. It provides a model in which component resources are isolated and their interactions are mediated via RPC. The framework supports multiple kernels as a foundation (including NOVA, formally verified seL4 [6], Fiasco.OC, and Linux), and provides a ready-made set of drivers, file systems, and services, allowing developers to focus on application logic without having to reinvent or port the underlying infrastructure. Access to resources in Genode is organized in accordance with the principles of capability-based security. In this context, a capability is a token granting the right to perform a specific operation on a specific object. A component can use a resource only if it possesses the corresponding capability, which was explicitly granted to it by a parent component. At the same time, the parent component is fully responsible for granting the appropriate capabilities to the child component and for passing its requests further up the hierarchy. This prevents unauthorized access to resources by bypassing the hierarchy, which differs significantly from the approach of the operating systems we are familiar with today, where a process with sufficient privileges can access any resource [7]. As part of the phantomuserland-snapper project [8], PhantomOS is being ported to Genode as a set of components. The Phantom Virtual Machine (PVM) runs as a Genode user process. To create, manage, and restore snapshots, snapper [9] was developed, which uses the file system for storage. This approach allows Phantom to use Genode's proven drivers and services without implementing them independently; the isolation of Genode components provides an additional layer of protection; and the use of microkernels

enables a trusted computing base (TCB) of modest size by today's standards, which also positively impacts the OS's security.

1.4 Problem statement

However, some security issues still need to be addressed. For instance, unlike most modern operating systems, which support disk encryption, PhantomOS does not support this feature in any form. A review of the literature also revealed that no research has been conducted on PhantomOS in this area. Porting does not solve the problem either, as Genode lacks ready-made, adapted disk encryption mechanisms. Moreover, the information flushed to disk at the moment a snapshot is created and stored there may be more sensitive than the files on the disk, since confidential data may be present in memory at the time the snapshot is taken. This is largely similar to the security issue with virtual machine snapshots. Under certain conditions, this could allow the state and subsequent operation of the machine to be fully reproduced on an attacker's device. This necessitates the implementation of protection for snapshots at rest. Thus, the focus of our research will be a mechanism for the secure storage of snapshots. The goal of this work is to ensure the confidentiality of PhantomOS snapshots at rest. This leads to the following objectives:

- Analyze computer usage scenarios and identify among them the secure scenarios that we will support;
- Develop or adapt a Genode component that implements such a scenario;
- Verify the correctness of the protection: demonstrate that the snapshot's contents are inaccessible;

- Evaluate the impact of encryption on performance by comparing the speed of operations with encryption enabled and disabled;

The scientific novelty lies in the limited research on this topic and the absence of any ready-made mechanisms for solving the identified problem. As of the time of writing, no components for Phantom similar to the one developed during the project se_vault have been found. The thesis is organized as follows: Chapter 2 provides a literature review on orthogonal persistence, the features of PhantomOS, and methods for protecting data at rest. Chapter 3 describes the methodology: the use case model, component requirements, and implementation plan. Chapter 4 is devoted to the adaptation of the file_vault component and related adaptations of adjacent components. Chapter 5 analyzes the results, limitations, and directions for further work. The conclusion is presented in Chapter 6.

Chapter 2

Literature Review

This chapter provides an overview of existing research related to our topic. The chapter is divided into four main sections, each of which will examine works pertaining to the following:

Section 1: The concept of orthogonal persistence and its implementation options;

Section 2: PhantomOS and its implementation of the concept of orthogonal persistence;

Section 3: Details and features of the PhantomOS port to Genode;

Section 4: Methods of protecting data at rest and their characteristics;

2.1 Persistence

2.1.1 Persistence definitions

Data persistence refers to the period of time during which data exists and is used [1]. This is precisely how some of the earliest researchers in the field—Morrison

R. and Atkinson M. P.—define this concept. In their work, they identify categories of persistence, divided into two groups:

1. Provided by the programming language;
2. Provided by the environment;

Researchers are striving to develop systems in which data usage is independent of the group or category to which the data belongs.

2.1.2 Orthogonal persistence principles

The concept of orthogonal persistence is based on principles formulated by researchers:

1. Independence;
2. Orthogonality of data types;
3. Persistence Identification;

In this case, independence means that data persistence does not depend on how the data is manipulated (the user cannot move data between long-term and short-term storage). The orthogonality of data types ensures that there are no cases in which data of any type cannot be persisted. The third principle means that persistent object identification does not relate to the type of such an object. A system that follows all three principles is orthogonally persistent.

2.1.3 Advantages of a persistent system

The desire to create such a system stems from the fact that it offers a number of advantages over conventional systems. These include [10]:

1. Increased programming productivity through simplified semantics;
2. Avoiding ad hoc solutions for data transformation and long-term data storage;
3. Providing security mechanisms for the entire environment;
4. Support for gradual evolution;
5. Automatically preserving referential integrity across the entire computational environment for the entire lifetime of an application.

All of these are, to some extent, the result of simplifying the memory management model for application programmers. This eliminates a huge amount of code that would otherwise be required to handle I/O and data storage. This also includes serialization/deserialization and initialization/deinitialization.

2.1.4 The cost of persistence

All of the advantages outlined above come at a cost. In addition to the general challenges associated with creating and implementing a new system—especially one built on principles that are entirely different from those we are accustomed to—there are also challenges specific to orthogonally persistent systems. These include:

1. The need for a stable object store: This depends largely on the specific implementation method, so we will return to this topic in Section 2.2.
2. Reduced performance of some applications: Because the programmer does not have full control over the location of the data, access speed may be reduced. A similar problem exists when using virtual memory.

3. The cost of providing language-independent binding mechanisms.

As will be discussed in Section 2.2, Phantom has solutions to minimize each of these problems. For now, note that point 1 depends largely on the implementation, and it is difficult to generalize information regarding it. Problems similar to point 2 also affect (albeit often to a lesser extent) systems with virtual memory, since the programmer does not have full control over the layout of the data, which can slow down access to it.

2.2 PhantomOS

2.2.1 PhantomOS Overview

PhantomOS is an open-source operating system. The concept was conceived by Dmitry Zavalishin, who also made the primary contribution to the system's development. Like other modern operating systems, Phantom features multitasking [11], [12], [13], a windowing subsystem [14], a network stack, and other familiar attributes. However, for our work, another feature is of greatest interest. PhantomOS provides an orthogonally persistent environment for applications within the Phantom Virtual Machine (PVM), where code is executed in the Phantom programming language. However, it is worth noting that there is also a POSIX compatibility subsystem. Access via arbitrary addresses is prohibited. Phantom already implements subsystems such as:

- Kernel itself: threads, synchronization, persistent memory management;
- Bytecode virtual machine - running native applications;
- POSIX layer - runs Linux-compatible (but not yet persistent) code;

- Graphics subsystem - Windows, controls, UI;
- Networking (TCP/IP);
- Phantom language compiler - the most native userland language;
- Java to Phantom translator - work in progress;
- Python to Phantom translator - just started;

All of this is specified in the documentation [4].

2.2.2 Mechanisms for ensuring persistence

Dmitry Zavalishin believes that it is impossible to preserve the state of the entire machine, but this is not necessary to achieve persistence [15]. Therefore, we will now discuss the mechanisms for ensuring a persistent environment. Persistence in PhantomOS is achieved by page-by-page mapping of the entire memory (PVM) to disk and periodically saving it. The kernel manages the snapshot process. It takes snapshots after first bringing the programs to a state where they are fully represented in memory. That is, all information from the processor registers is also transferred for this purpose. Application programs are blocked only during this brief preparation phase. The disk is switched to read-only mode during this process, but this does not cause a lock throughout the entire process of writing the snapshot to disk thanks to the use of CoW: if a page has been written to the snapshot, nothing prevents access to it; otherwise, a copy is created, which the program operates on, and the original page is moved closer to the front of the save queue [15], [16]. Writing the entire memory to disk with every snapshot is a very costly operation that is unnecessary. To optimize this process, only the increment

is written to disk, and preemptive writing is also used. The latter, by the way, serves two purposes:

- To speed up the snapshot writing process;
- To meet the increased demand for memory during the snapshot;

2.2.3 System and Persistent Environment features

One of the goals behind the system's design is to make life easier for programmers. They no longer need to manage memory; instead, the garbage collector handles this task. Other key features of the system include a global address space and the requirement that objects can only be accessed via a reference. Access via arbitrary addresses is prevented, which has a positive effect on security. It is in this environment that the garbage collector operates. The large amount of virtual memory and the inability to use stop-world algorithms led to a strategy of using two garbage collectors [17]:

1. Partial;
2. Full;

The first is fast, inexpensive, and possibly incomplete. It is assumed that it cleans up garbage in RAM without waiting for objects to be flushed to disk. It runs continuously and is currently implemented based on counting the number of references to an object. The second should be full, but it can run periodically thanks to the presence of the first. At the same time, a stop-world implementation is also possible due to the persistence of garbage: that is, what was garbage in the old snapshot will remain garbage in the new one. Based on this idea, it is proposed to perform garbage collection on the old snapshot, although not the entire snapshot

may be used for this, but only a reflection of its memory state. However, there are still a number of problems that need to be solved before its implementation [18].

2.3 OS Phantom to Genode port

2.3.1 Reason for porting

As previously mentioned, PhantomOS is currently being ported to the Genode framework. This is being done to improve [19]:

- Reliability;
- Security;
- Interoperability;

In this way, we plan to achieve the required quality. One of the main reliability issues—the new kernel with built-in drivers—is planned to be resolved by using kernels supported by Genode, including many microkernels known for their high reliability. It is also proposed to replace new, insufficiently tested components with already proven ones offering the same functionality. This will also resolve the issue of interoperability, as there will be no need to rewrite existing components, and the focus can shift to the system's logic. The Genode developers have taken a thorough approach to security. The framework uses a capability-based security system. Microkernels are often used as the system kernel. This makes it possible to isolate components to a significant degree and configure access to resources more flexibly, allowing for a minimal TCB. This results in a minimal attack surface, which has an extremely positive effect on reliability.

2.3.2 Affected components

Initially, work was carried out to port the system [19]. Essentially, the task involves migrating the persistent environment to the framework. In addition to PVM, work was done on HAL, libc, and the physical memory allocation mechanism. Some functions were replaced with placeholders. The result of this work was a working isomem project [20]. It also includes a video placeholder in the form of a window subsystem with limited functionality. It is configured and runs as a separate, standalone Genode component. Subsequently, work was carried out on developing a persistent network subsystem for the [21] port and implementing the WASM bytecode runtime [22]. The snapshot creation mechanism was also modified, resulting in the creation of the snapper project [9].

2.3.3 Snapper

The current version of Snapper (Snapper 2.0) is designed to address the issues encountered in the implementation of the previous mechanism—the “superblock” [23], [24]. It improves disk space efficiency when storing multiple snapshots. Previously, snapshots were stored as monolithic objects in memory, which resulted in a large number of data copies when storing multiple snapshots. For unchanged pages, Snapper adds a link in the new generation without copying all the data again. The ability to recover from failures is ensured by using file systems for which such tools already exist, for example: ext4 and fsck. Using a file system to store snapshots can be more convenient during development and debugging. Integrity checks have been added. In addition, the system has become configurable in terms of reliability through the introduction of a parameter called redundancy. If the specified number of links to a file is exceeded, a copy of the file is created, and

all generations using this file are linked to both the original file and its copy. This allows an experienced administrator to independently configure the balance between reliability and storage usage.

2.4 Data protection at rest

Snapper has helped improve snapshot security by adding integrity checks, but neither it nor any other component ensures secure storage. They are stored on disk in plaintext, which can pose a serious security risk in many device usage scenarios.

2.4.1 Threats to stored data

Data at rest refers to any data stored on persistent storage media that is not currently being actively transmitted or processed. It is static and vulnerable to threats related to physical or logical access. Sensitive data may be stored on the disk and could be read or overwritten. In the context of PhantomOS, the situation is complicated by the fact that a snapshot is taken regardless of what data is in memory, and secret data may end up in it, even if the application level of the program provides for its protection using its own means [25]. This is similar to the problem of protecting virtual machine snapshots [26].

2.4.2 Ways to protect stored data

Data protection methods must align with the threat model, which, in turn, depends directly on the device's usage scenarios. This subsection provides only a brief overview of such methods, without a detailed explanation of why a particular method might be chosen. In this case, we are discussing storage encryption. We

do not consider options involving robust physical and/or logical access controls. Encryption can be classified by scope [27]:

- Full disk encryption;
- Volume and virtual disk encryption;
- File and folder encryption;

and by level [28]:

- Block level;
- File system level (generalized, divided into several points regarding the number and removal of file systems);
- Application level;

These are the levels used at the OS level and above. Each higher level leaves the metadata of the lower level exposed. In addition to these, implementation at a low-level hardware level is possible, which would be transparent to the OS. Due to the absence in Genode of verified systems of this type or tools for their creation, this level will not be considered.

2.4.3 Current work

In this project, we will focus on implementing an environment for the secure storage of snapshots. This will be a block-level encrypted storage solution, which will enhance security compared to file-system-level encryption and higher, by hiding metadata and making it more difficult to extract system information. This is also facilitated by the fact that Genode already has a library [29] for this purpose and a test component [30], [31].

Tresor performs AES encryption on 4K-sized blocks. The virtual block device key is encrypted with a master key, which is managed by the Trust Anchor. Copy-on-write (CoW) is used during writing, ensuring the atomicity of the operation and improving the ability to recover from failures. The component uses SHA-256 for data integrity checking.

Chapter 3

Methodology

This chapter discusses system requirements and the implementation plan. First, we will describe a model of computer usage and identify the secure scenarios we intend to support. Next, we will outline the requirements for the component. After that, we will explain which component will be adapted and how.

3.1 Use cases

To identify the most appropriate requirements for the component, it is necessary to define the device's usage scenarios. To this end, a diagram of computer usage scenarios was created, with unsafe and safe scenarios marked on it. From among the safe scenarios, those that are supported and can be implemented within the scope of this project were selected.

3.1.1 All use cases

Before describing the diagram in Fig. 3.1, it is worth mentioning what it depicts and how it was constructed. The diagram is a graph whose cycle begins

and ends with the “Power off” state. It illustrates the entire computer operation cycle. Each possible path represents a distinct usage scenario. When creating it, we aimed to describe absolutely all possible computer usage scenarios, grouping them according to criteria that were important to us. The grouping was done without partial overlaps or complete overlaps; actual operation may, and often will, be described by combinations of such scenarios. States on which the subsequent state or the security of the entire path does not depend were excluded for simplicity. Cases that are a priori incorrect were also not considered, such as receiving a valid authentication factor from an untrusted user.

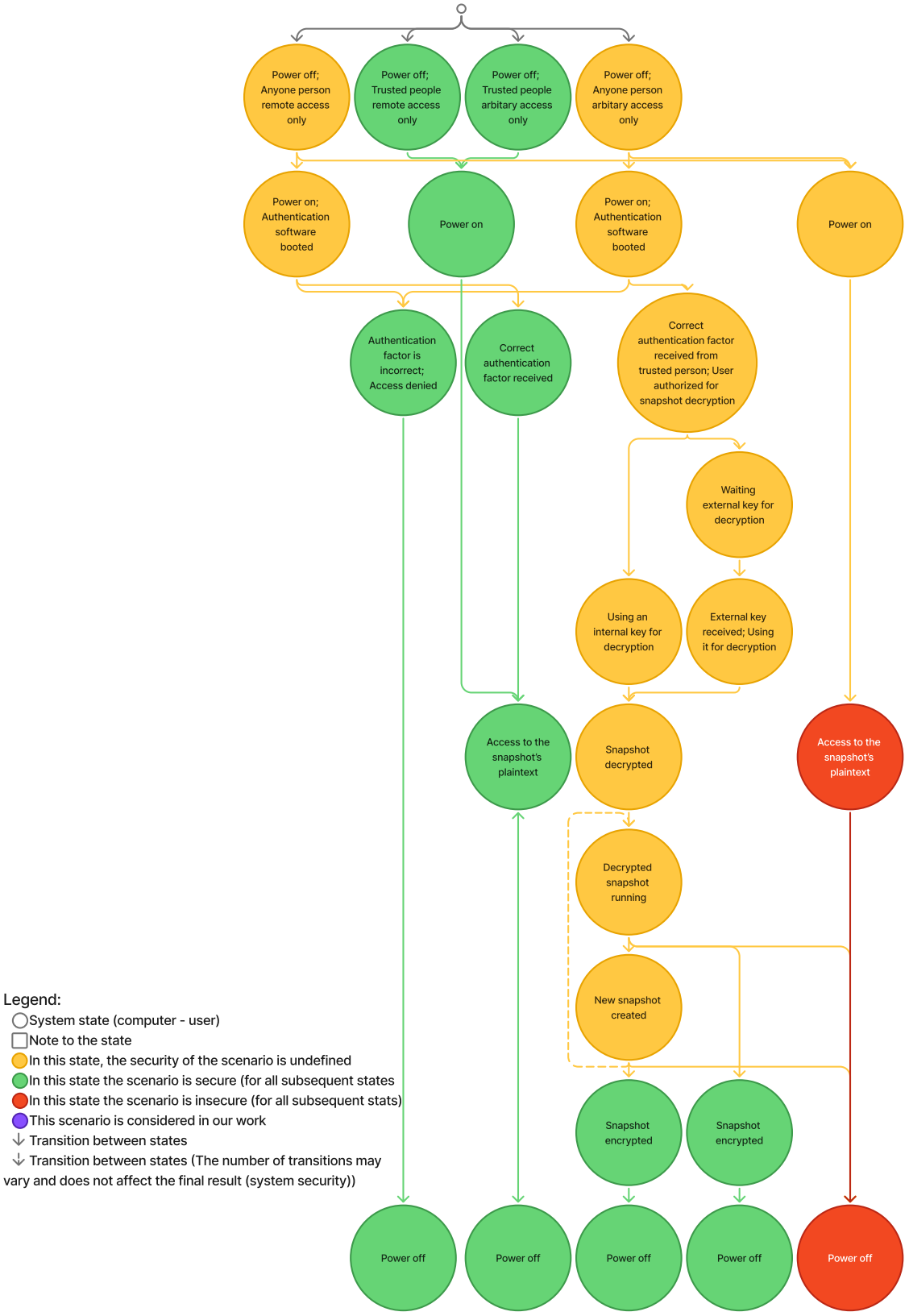

Fig. 3.1. Use case scheme

First, four initial states were identified. These were derived by classifying the states based on access method and user privileges. In this context, remote access refers to the ability to interact with the computer exclusively through trusted input/output devices. Arbitrary access refers to the absence of any restrictions on interaction. In fact, the set of remote access options is a subset of the set of arbitrary access options. Of course, security issues at the hardware level can lead to system compromise. Exploits below the OS level (hardware, firmware) are not considered in this work and do not affect the security assessment of the scenario. The same applies to users. A trusted user is one whose access to the computer is legitimate. By and large, the task boils down to ensuring that, of all users, only trusted ones can gain access.

Scenarios involving only trusted methods are simplified as much as possible, since with such a separation, the task at hand is always accomplished. To identify trusted users, we use authentication software that we consider reliable. We also believe that authentication factors are selected in such a way that their validity unambiguously identifies a trusted user. Among the remaining options, those without authentication and remote access are significantly simplified.

3.1.2 Supported use cases

For our work, we selected secure authorization scenarios in which all users have unrestricted access to the device. It is illustrated in Fig. 3.2. These scenarios differ from insecure ones in that they guarantee the snapshot remains encrypted upon completion of the operation. We also note that scenarios in which, after granting access to a trusted user, an untrusted user gains access are not considered. In other words, every user, without exception, goes through authorization. The diagram omits details regarding the choice of authentication factor and encryption

type. The use of external storage for the key is due to the simplicity of implementation while maintaining a security level acceptable to us. This will allow us to combine authentication with decryption.

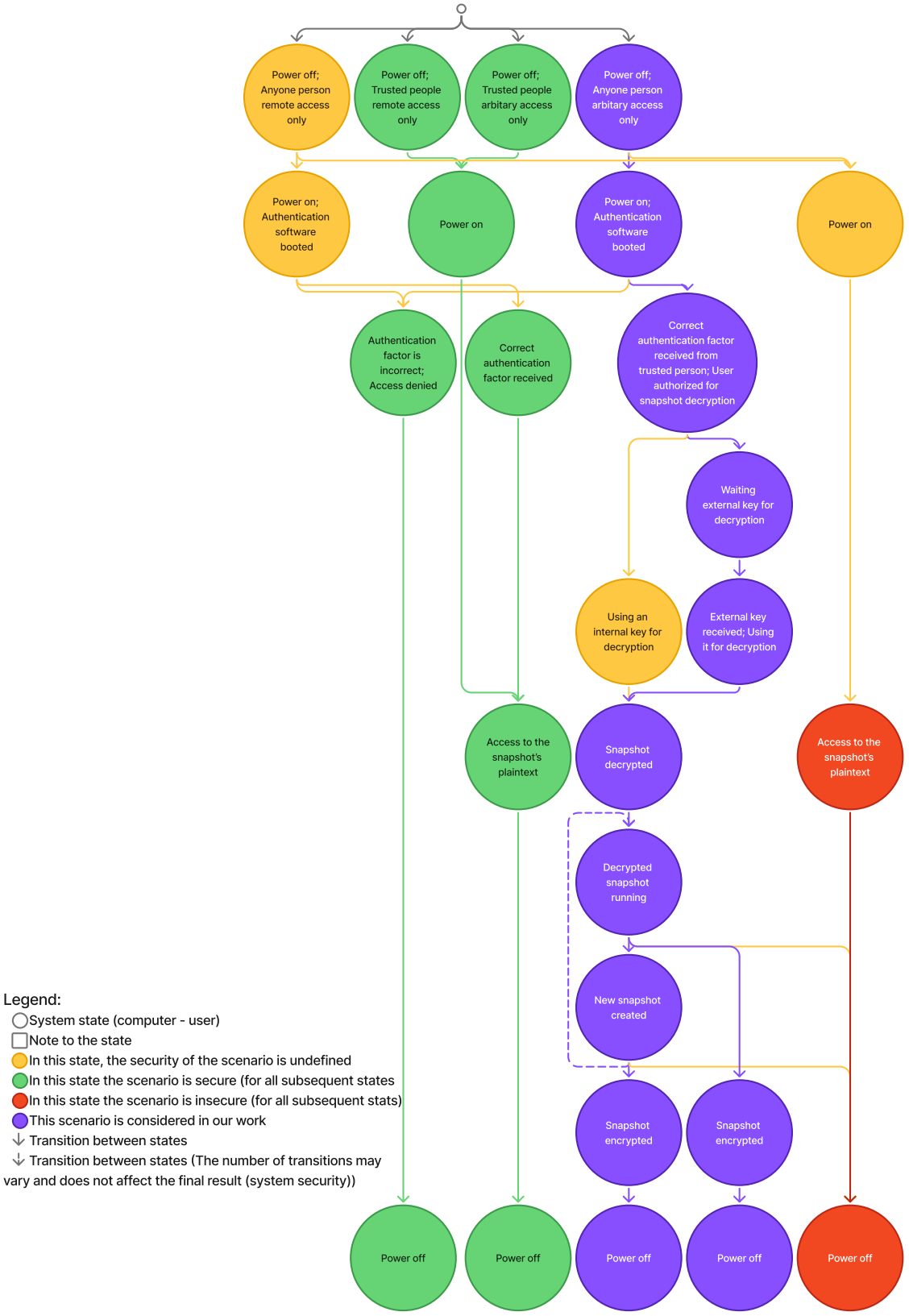

Fig. 3.2. Supported use case scheme

3.2 Requirements

The scenario dictates the following implementation requirements:

1. Mandatory authorization;
2. Storage of keys on an external medium;
3. Encryption of snapshots;
4. Guarantee that only encrypted snapshots are stored;
5. Acceptable performance loss of no more than 2 times;

The first two requirements can be combined. Since a long key is required for reliable encryption, a USB drive containing it will serve as a secure factor that is difficult to forge. This will simplify the process without compromising the quality of the result. As mentioned earlier, Genode includes a library for block-level encryption—Tresor. It will enable the fulfillment of the last two requirements. Encryption will occur upon writing each block, ensuring that only encrypted data is written to the disk.

3.3 Implementation plan

Genode already includes a file_vault component used to create secure storage. The Tresor library is used for encryption. We plan to adapt this component for encrypting snapshots. To do this, all unnecessary functionality—mostly related to user interaction via the graphical interface—will be removed. The ability to recover the file system using fsck will also be added. The ability to properly shut down via the Phantom GUI will be added, as currently there is only a placeholder

for this. For other system components, this should be transparent and require no changes. In addition, optimization work will be carried out, and the component will be configured to use keys from a USB flash drive.

Chapter 4

Implementation

In this chapter, we will examine the implementation of the component, discuss its testing, and present the results. In Section 4.1, we will examine the structure of the component in question and describe the changes made. Section 4.2 will focus on the adaptation of other components that was necessary to ensure the functionality of our solution. In Section 4.3, we will describe the testing methodology and the results of the tests themselves.

4.1 File_vault adaptation

In this project, our goal is not to create a universal tool or preserve all the functionality of the target system; therefore, during the adaptation process, we will retain and add only the necessary features. The modified component is hereinafter referred to as se_vault[1].

File_vault manages the configuration of the components responsible for encryption. It is a state orchestrator in the sandbox that provides File_system clients with a secure storage session, the sole client of which in our case will be snapper.

[1] https://github.com/VoronM1522/genode/tree/se/repos/gems/src/app/se_vault

Upon startup, it prompts the user for a password. If the password matches, it determines the storage's state and either initializes it or unlocks it. Before shutting down, it seals the component, which resets all blocks and ensures the storage's consistent state. file_vault itself receives and routes to child (orchestrated) components the file system sessions containing the secure vault file image and information belonging to the Trust Anchor. Initialization occurs automatically. A passphrase may be requested, but this behavior is configurable.

The UI was deemed redundant, as Snapper uses only the API. The UI can also be used to enter a passphrase or request a file system resize. In our case, authentication is handled via a USB drive containing a key (and other Trust Anchor data), and there is no need to resize the file system during operation; therefore, the UI can be completely eliminated. This led to a significant reduction in the amount of code and the number of states being handled. In addition, all code for generating fs_query components was removed, since monitoring the return codes of the components proved sufficient for correct state transitions.

Next, an option was added to check and restore the FS state using fsck. Initially, file_vault created an ext2 filesystem. It was replaced with ext3, and later with ext4. The reason for this was the lack of journaling, which could cause problems during recovery after crashes. The rump_fs component also drew attention, as it lacks the ability to configure the cache size, which is necessary to achieve a balance between reliability and speed. It was replaced with lwext4[2].

[2]https://github.com/VoronM1522/genode-wundertuete

4.2 Related adaptations and optimizations

After replacing rump_fs with lwext4, problems arose when creating a file of the desired size. It turned out that *truncate_file* does not check the file size after creation, and lwext4 creates a file of a smaller size. The problem was resolved by adding a pointer cast to the *ftruncate* function in lwext4.

After that, the system started up, but it crashed upon restart, reporting numerous hash mismatch errors. Attempts to reduce the cache size or disable it yielded no results. The problem was later identified and fixed. The issue was that tresor updates the hash during a call to *sync*. When *sync* is called, lwext4 simply flushes the contents of the buffers to disk. A block of code was added that passes the *sync* call to the lower levels of the VFS, allowing tresor to correctly receive this call and update the state hash in a timely manner.

At the time the work described above was completed, the system still lacked a standard shutdown procedure, which meant that Tresor never flushed all blocks explicitly; consequently, there was a risk of losing the latest generation in the event of a failed shutdown. This issue was resolved by making changes to isomem[3], snapper[4], and vfs[5]. A callback for the power-off button was added to isomem. After the button is pressed, the system sets a flag indicating readiness to shut down. This triggers the creation of the final snapshot either immediately (while waiting for a snapshot) or after the current snapshot is completed, after which isomem releases CPU resources but does not signal its termination to the system. To fix this, the termination process was replaced with a call to *env.parent().exit(0)*. Client counters were added to snapper and vfs; these trigger a shutdown when their count reaches zero. However, a configuration flag was also added to vfs for

[3]https://github.com/VoronM1522/phantomuserland-snapper
[4]https://github.com/VoronM1522/snapper
[5]https://github.com/VoronM1522/genode/tree/se/repos/os

shutdown, as this counter would reset during state transitions, causing vfs to shut down prematurely. In addition, the vfs shutdown process was modified so that all connected plugins would shut down correctly. This was necessary to unmount lwext4 and set the corresponding FS flag. Otherwise, when using fsck, the next system boot would start with a check. A counter was also added to se_vault, but after all clients were disconnected, the vault was sealed and a flag was set for the child VFS. Only after this does se_vault terminate.

Next, optimization was performed. Looking at *file_vault*, it is easy to see that the component named *tresor_vfs* accesses the file image using a block session obtained from the *block* VFS plugin. This was likely done for the convenience of configuring and resizing the storage. However, a fixed size that does not exceed the size of the available disk will suffice for us. Therefore, we can remove the code for creating the image file and pass the block session directly. We use a similar approach for the Trust Anchor storage: instead of the File_system session, we pass the block session to the USB.

We have not forgotten that Tresor works with 4K blocks. To ensure better consistency and reduce the overhead of managing blocks, all of them (at the block level and at the file system level) have been standardized to this size.

4.3 Tests and results

The OS was booted in QEMU, using files and a USB flash drive as storage media[6]. A 2 GB image was used for isomem, and a 5 GB image for se_vault (the user file system was 4 GB, with 1 GB reserved for Tresor's internal structures).

First, the functional part was tested. To verify operability, the OS was booted,

[6]Run scripts and system configuration: https://github.com/VoronM1522/phantomuserland-snapper

a snapshot was written to disk, after which the process was terminated and the OS was booted again. The success criteria were error-free read/write operations and the ability to restore from the snapshot upon reboot. No issues were observed with this.

Next, the encrypted image and the key generation mechanism were checked in a straightforward manner. After completing operations at various stages (both normal and emergency), an attempt was made to mount the image or detect the file system using fsck. All tests were passed successfully. This means that it is not possible to extract information from the storage medium in this way. At the same time, images are visible in the image used without encryption. The possibility of replacing the key and superblock hash was also tested. To do this, after the process was completed, files were transferred from the USB drive, the main storage device's image was changed, a new boot was performed to generate keys, after which the old image and hash were restored, and the system was booted. In this case, the superblock is not detected, which can also be considered a success.

Similar to the functional testing, the standard shutdown and the optional recovery using fsck were tested. We verified that when the shutdown button is pressed, a snapshot is taken, isomem is verified, followed by snapper; the file system is unmounted and vfs is terminated; the secure storage is locked; and se_vault is terminated. All stages were completed without issues. Afterward, it was verified that the file system mount flag was not set upon a new boot, which was also achieved. To test recovery with fsck, the added option *fsck_apply="no"* was set in the component configuration. After that, the system was booted and crashed. The test was considered passed if, upon a restart, fsck ran, fixed the errors, and the system operated normally. After changing the block sizes, fsck reported successful error correction with code "1"; however, during further operation, lwext4 errors

related to *bcache* appeared. The problem was resolved by applying the *"-B 4096"* option. After that, no further issues were observed.

Finally, speed tests were conducted. The plan was to measure and compare the time taken to create each of the first snapshots for both the encrypted and unencrypted versions, as well as the restore speed during bootup. Results were considered acceptable if performance did not decrease by more than a factor of 2, which represents a very significant overhead for encryption. Unfortunately, the very first result was striking, showing a time increase of more than a hundredfold for the first snapshot. Part of the problem lay in the standard QEMU options for PC, which use *-cpu Nehalem-v2*. Because of this, AES and SHA hardware acceleration did not work. After switching to *-cpu host*, performance improved significantly, but it was still very low. Attempts were made to replace the ahci driver with nvme, which slightly improved performance. Options were tested without a USB flash drive (using an image accessible via nvme instead), with a reduced tree size in tresor, but this also failed to provide the desired improvement. The final results are as follows: the first snapshot is created without encryption in ≈2.1 s, and with encryption in ≈51 s (see Table 4.1); restoration takes ≈2.5 s and ≈27.3 s, respectively (see Table 4.2). It later turned out that the Tresor performance issue is known and has not yet been resolved[7]. It lies in the library's implementation itself, which is not optimized for multithreading. Some significant improvements were achieved only in the context of initialization compared to the original file_vault, since the current implementation skips this step entirely. Previously, creating a large file image took a long time. For example, an image for a 4 GB user file system took about 20 minutes to create. The current implementation skips this

[7]Genode Users Mailing List, 2022: https://www.mail-archive.com/users@lists.genode.org/msg02999.html; https://lists.genode.org/mailman3/hyperkitty/list/users@lists.genode.org/thread/YJH7QFWGLHBKPP5VMCDUAIM6IIY7C6MC/

step entirely.

TABLE 4.1
First snapshot creation time over 5 runs (seconds)

Run	Without encryption	With encryption	Overhead (times)
1	2.08	50.84	24.44
2	2.09	50.82	24.32
3	2.13	51.19	24.03
4	2.11	51.61	24.46
5	2.09	50.54	24.18
Mean	**2.10**	**51.00**	**24.29**

TABLE 4.2
Snapshot restore time over 5 runs (seconds)

Run	Without encryption	With encryption	Overhead (times)
1	2.46	27.52	11.19
2	2.49	27.52	11.05
3	2.53	26.98	10.66
4	2.47	27.20	11.01
5	2.55	27.28	10.70
Mean	**2.50**	**27.30**	**10.92**

Chapter 5

Evaluation and Discussion

In this chapter, we will take a closer look at the results of our work. We will examine the findings, limitations, and shortcomings of our study, as well as ways to address them. Finally, we will touch on future research in this area.

5.1 Results

During the course of the research, the file_vault component was adapted for use by Snapper as a snapshot storage system. The integration was seamless for Snapper and other components; only minor changes were required to ensure proper functionality, changes that would have been necessary even without our component. Only a few modifications to lwext4 can be considered truly necessary specifically for this project. A comparative analysis of the resulting and target components can be seen in Table 5.1.

TABLE 5.1
Comparison of file_vault and se_vault

Parameter	**file_vault**	**se_vault**
Purpose	General-purpose secure file storage	Snapshot encryption for Phantom OS
User interface	GUI via ui_config / ui_report ROM	None (headless)
Authentication	Passphrase entered at runtime	Key file read from USB drive
File system driver	rump_vfs (ext2)	lwext4 (ext4)
fsck support	No	Yes (configurable via fsck_apply)
Storage backend	Image file via File_system session	Direct block device session
Trust Anchor storage	File_system session	USB block session
Online extend / rekey	Yes	Removed
Image file creation	Pre-allocated via truncate_file	Not applicable
Block size	512	Standardized to 4 KB
Init time (4 GB FS)	≈20 min	≈1 min

A broader comparison of the two configurations—snapshots stored without encryption versus snapshots protected by se_vault—is presented in Table 5.2.

TABLE 5.2
Comparison of unencrypted and encrypted snapshot storage

Property	**Without encryption**	**With encryption (se_vault)**
Confidentiality at rest	None; snapshots stored in plaintext	AES block-level encryption
Integrity protection	FS level (Snapper's hash)	FS level (Snapper's hash) + Block level (Tresor's SHA-256)
Authentication	None	Trust Anchor Data Ownership
Unauthorized data extraction	Possible; image mountable directly	Not possible
Recovery support	Applicable; Not configured yet	ext4 with fsck (configurable)
Snapshot creation time	≈2.10 s	≈51.00 s (≈24 (times) overhead)
Snapshot restore time	≈2.50 s	≈27.30 s (≈11 (times) overhead)
Storage initialization (4 GB)	Not required	≈1 min

Of the five requirements stated in Section 3.2, four were met; requirement 5 (performance loss no greater than $2\times$) was not met due to the known performance

limitations of the Tresor library. Although we managed to secure the snapshots, performance has degraded significantly. This may not be as critical for less demanding tasks, since snapshots do not block Phantom processes for extended periods of time. For subsequent snapshots, the situation is somewhat different: creating a snapshot involves reading hashes to locate changed pages and writing the changed pages. Since the overhead for reading is significantly lower than for writing, the total overhead is determined by the ratio of modified to unmodified pages and depends largely on the usage scenario, which is difficult to verify on a demo version of the OS. On the other hand, Dmitry Zavalishin described a problem where a large amount of memory is used and becomes obsolete between snapshot creations, citing rendering computation as an example [18]. When using the OS for similar tasks where a relatively long interval between snapshots is required and data loss during that period would not be critical, it is possible to mitigate the consequences to a significant extent. This is best verified through practical use, but that is currently impossible due to the system's lack of readiness.

5.2 Limitations

This project certainly has a number of serious limitations. First, Tresor is not intended for use in a production environment. The author himself acknowledges this. The problem is that it is the only library designed for block-level encryption. Since the author left the team, support for the project has deteriorated even further. Second, testing was conducted on small amounts of user space. It cannot be stated with certainty that the system will perform with comparable levels of performance and stability as the volume of data increases exponentially. Third, testing was conducted only on a virtual machine with placeholders instead of many real working

components. Although this scenario provides an idea of approximate results, it still differs significantly from real-world usage and is also influenced by the state of the host system. Finally, the measurements taken are not pure I/O performance tests: when creating the first snapshot, only a write operation to the secure storage is performed, and during restoration, only a read operation from it; however, in the first case, a read operation from isomem is performed, and in the second, a write operation to it. The results should be viewed as an estimate of the overhead for encryption for each snapshot operation individually, rather than as an overall assessment of system performance.

5.3 Shortcomings

The performance issue is obvious, as has been pointed out many times. The only apparent solution is to make corrections to the library. Another approach is to sacrifice security to some extent—though the exact extent remains to be determined—and implement a simpler encryption method. This can be done at the file system level. Genode includes an OpenSSL port, so there is no need to implement cryptographic algorithms from scratch, which would be unacceptable.

5.4 Future work

Clearly, we should focus on improving performance in the future. However, there is another aspect that has not been discussed yet. It is worth noting that isomem also uses a block device. This is part of the persistent storage where pages are swapped out from RAM. When the system shuts down, a large number of artifacts remain in it, allowing the machine's state at the time of shutdown to be

almost completely restored. This should be avoided. It is not difficult to modify se_vault to provide a block session instead of a filesystem session. This will allow it to be used for encrypting the contents of the isomem block device. Moving the file system outside the sandbox will increase the solution's flexibility. For greater security, conversely, one can place snapper and/or isomem inside the sandbox. This is a fairly simple solution to implement. In the early stages of adaptation, this option was also used in our work, but we abandoned it in favor of configurability. If it is absolutely necessary to protect data, it is still possible to do so using the *block* VFS plugin, but given the current performance levels, this is undesirable. Another approach could be to attempt to combine snapper and tresor. Tresor also stores snapshots and uses incremental updates when writing. In this regard, their functionality is similar, but the components use it differently.

Chapter 6

Conclusion

During this project, we investigated methods for securing PhantomOS snapshots. To this end, we adapted the component responsible for encrypting snapshots. In addition, we made modifications to other components to ensure that se_vault functioned correctly.

The objectives outlined in the research were partially achieved: snapshot security was successfully ensured, but this had a significant impact on performance.

In the future, this should aid in researching the security of persistent operating systems in general and the PhantomOS port on Genode in particular. Work of this kind should have a positive impact on the capabilities of such operating systems and may contribute to their faster adoption.

The research direction has not yet been exhausted; future work will focus on optimizing the resulting component and expanding its scope of application within the context of this operating system.

Bibliography cited

[1] R. Morrison and M. P. Atkinson, "Persistent languages and architectures," in *Security and Persistence*, J. Rosenberg and J. L. Keedy, Eds., Springer-Verlag, 1990, pp. 9–28.

[2] A. C. Bomberger, W. S. Frantz, A. C. Hardy, N. Hardy, C. R. Landau, and J. S. Shapiro, "The KeyKOS nanokernel architecture," in *Proceedings of the USENIX Workshop on Microkernels and Other Kernel Architectures*, Seattle, WA, 1992, pp. 95–112.

[3] E. Tsalapatis, R. Hancock, T. Barnes, and A. J. Mashtizadeh, "The Aurora single level store operating system," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*, ACM, 2021, pp. 788–803. DOI: 10.1145/3477132.3483563

[4] D. Zavalishin, *Phantom os documentation*, Available: https://app.readthedocs.org/projects/phantomdox/downloads/pdf/latest/, 2020.

[5] Д. Завалишин, "Операционная система «фантом»," *Открытые системы*, 2011. [Online]. Available: https://www.osp.ru/os/2011/03/13008200

[6] G. Klein et al., "seL4: Formal verification of an OS kernel," in *Proceedings of the 22nd ACM SIGOPS Symposium on Operating Systems Principles*

(SOSP), Big Sky, MT, USA: ACM, 2009, pp. 207–220. DOI: 10.1145/1629575.1629596

[7] N. Feske, *Genode operating system framework foundations*, Available: https://genode.org/documentation/genode-foundations-25-05.pdf, 2025.

[8] R. Mitov, *Phantomuserland-snapper*, Available: https://github.com/rumenmitov/phantomuserland-snapper.

[9] R. Mitov, *Snapper*, Available: https://github.com/rumenmitov/snapper.

[10] A. Dearle, G. N. C. Kirby, and R. Morrison, "Orthogonal persistence revisited," *arXiv preprint*, 2010, arXiv:1006.3448. [Online]. Available: https://arxiv.org/abs/1006.3448

[11] Д. Завалишин, *Делаем мультизадачность*, Хабр, Дата обращения: 2025, Apr. 2016. [Online]. Available: https://habr.com/ru/articles/282037/

[12] Д. Завалишин, *Преемптивность: Как отнять процессор*, Хабр, Дата обращения: 2025, Apr. 2016. [Online]. Available: https://habr.com/ru/articles/282049/

[13] Д. Завалишин, *От шедулера к планировщику*, Хабр, Дата обращения: 2025, Apr. 2016. [Online]. Available: https://habr.com/ru/articles/282213/

[14] Д. Завалишин, *Ос фантом: Оконная подсистема — делаем контролы*, Хабр, Дата обращения: 2025, Nov. 2019. [Online]. Available: https://habr.com/ru/articles/472160/

[15] Д. Завалишин, *Персистентная оперативная память*, Хабр, Дата обращения: 2025, Mar. 2016. [Online]. Available: https://habr.com/ru/articles/279443/

[16] D. Zavalishin, *Persistentmemory*, Available: https://github.com/dzavalishin/phantomuserland/wiki/PersistentMemory, 2019.

[17] Д. Завалишин, *Сборка мусора в персистентной модели: От терабайта и дальше*, Хабр, Дата обращения: 2025, Apr. 2016. [Online]. Available: https://habr.com/ru/articles/281236/

[18] Д. Завалишин, *Фантом: Большая сборка мусора*, Хабр, Дата обращения: 2025, Jun. 2017. [Online]. Available: https://habr.com/ru/articles/331974/

[19] A. Antonov, "Persistency in microkernel os: Porting experience of phantom os to genode os framework," Bachelor's thesis, Innopolis University, 2021.

[20] A. Antonov, *Isomem*, Available: https://github.com/S7rizh/phantomuserland/tree/15ea4c963c497255f824ffabd7e0cf987d3b45c5/phantom/isomem, 2024.

[21] A. Brisilin, "Networking in orthogonally persistent operating systems," Bachelor's thesis, Innopolis University, 2022.

[22] K. Samburskiy, "Introducing bytecode runtime into a persistent operating system," Bachelor's thesis, Innopolis University, 2024.

[23] R. Mitov and A. Tormasov, *Practical persistence on microkernels (ft. PhantomOS)*, FOSDEM 2026, Microkernel and Component-Based OS devroom, Дата обращения: 2025, Feb. 2026. [Online]. Available: https://fosdem.org/2026/schedule/event/DJ8YVT-practical-persistence/

[24] R. Mitov, "Snapper (v2): A PhantomOS snapshot mechanism," FOSDEM 2026, Tech. Rep., 2025. [Online]. Available: https://fosdem.org/2026/events/attachments/DJ8YVT-practical-persistence/slides/267604/snapper-v_5dptkby.pdf

[25] M. Carlson, *Persistent memory security threat model*, Flash Memory Summit 2018, 2018. [Online]. Available: https://files.futurememorystorage.com/proceedings/2018/20180807_PMEM-101-1_Carlson.pdf

[26] B. Liang et al., "Security issues and challenges for virtualization technologies," *ACM Computing Surveys*, 2020. [Online]. Available: https://web.engr.oregonstate.edu/~benl/Publications/Journals/ACM_Computing_Surveys_2020.pdf

[27] K. Scarfone, M. Souppaya, and M. Sexton, "Guide to storage encryption technologies for end user devices," National Institute of Standards and Technology, Tech. Rep. SP 800-111, 2007. DOI: 10.6028/NIST.SP.800-111 [Online]. Available: https://csrc.nist.gov/pubs/sp/800/111/final

[28] C. P. Wright et al., *Cryptographic file systems performance: What you don't know can hurt you*, Proceedings of the 2003 IEEE Security in Storage Workshop, 2003. [Online]. Available: https://www.filesystems.org/docs/nc-perf/index.html

[29] M. Stein, *Tresor (readme)*, Available: https://github.com/genodelabs/genode/blob/sculpt-25.04/repos/gems/src/lib/tresor/README, 2024.

[30] M. Stein, *File vault (readme)*, Available: https://github.com/genodelabs/genode/blob/sculpt-25.04/repos/gems/src/app/file_vault/README, 2025.

[31] M. Stein, *Tresor (readme)*, 2021. [Online]. Available: https://genodians.org/m-stein/2021-05-17-introducing-the-file-vault

Appendix A

Implementation

A.1 se_vault configuration example

```
<start name="se_vault" caps="1500" ram="2000M">
 <exit propagate="yes"/>
 <provides>
  <service name="File_system"/>
 </provides>
 <config
 jitterentropy_available="yes"
 fsck_apply="yes">
  <vfs/>
 </config>
 <route>
  <service name="Block"
  label_prefix="se_tresor_trust_anchor_vfs ->">
   <child name="usb_sec"
```

```
 identity="se_vault -> trust_anchor"/>
</service>
<service name="Block"
label_prefix="se_tresor_init ->">
 <child name="nvme2"
 identity="se_vault -> data"/>
</service>
<service name="Block"
label_prefix="tresor ->">
 <child name="nvme2"
 identity="se_vault -> data"/>
</service>
<service name="Block"
label_prefix="se_tresor_vfs ->">
 <child name="nvme2"
 identity="se_vault -> data"/>
</service>
<service name="Block"
label_prefix="init_flag ->">
 <child name="nvme2"
 identity="se_vault -> data"/>
</service>
<service name="Timer">
 <child name="timer"/>
</service>
<service name="Report">
```

```
    <child name="report_rom"/>
    </service>
  <service name="PD">
    <parent/>
  </service>
  <service name="ROM">
    <parent/>
  </service>
  <service name="CPU">
    <parent/>
  </service>
  <service name="LOG">
    <parent/>
  </service>
  <service name="RM">
    <parent/>
  </service>
  <any-service>
    <parent/>
  </any-service>
 </route>
</start>
```